

**Department of Electronics and Telecommunication Engineering**  
**(NBA ACCREDITED)**  
**Digital System Design Laboratory**  
**Academic Year 2025-2026, SEM-5, Odd Semester**

|                                    |                             |
|------------------------------------|-----------------------------|
| <b>Course Code</b>                 | EC24P                       |
| <b>Subject Professor In-charge</b> | Prof. Satendra Mane         |
| <b>Student Name</b>                | Rahul Bhadane               |
| <b>Roll Number</b>                 | 23104B0003                  |
| <b>Class</b>                       | T.E – EXTC/EXCS SEM-V       |
| <b>Division</b>                    | B                           |
| <b>Date of Performance</b>         | 25 <sup>th</sup> July 2025  |
| <b>Date of Submission</b>          | 1 <sup>st</sup> August 2025 |

**EXPERIMENT NO.2**

**Estimated time to complete this experiment:** 02 hours.

---

**Objective:** To simulate and Implement Half Adder & Half Subtractor using all methods of modeling.

---

**CO to be achieved:** CO1, CO2

---

**Expected Outcome of Experiment:**

- Students will learn to design, simulate and synthesis the arithmetic circuit using VIVADO & VERILOG.
- 

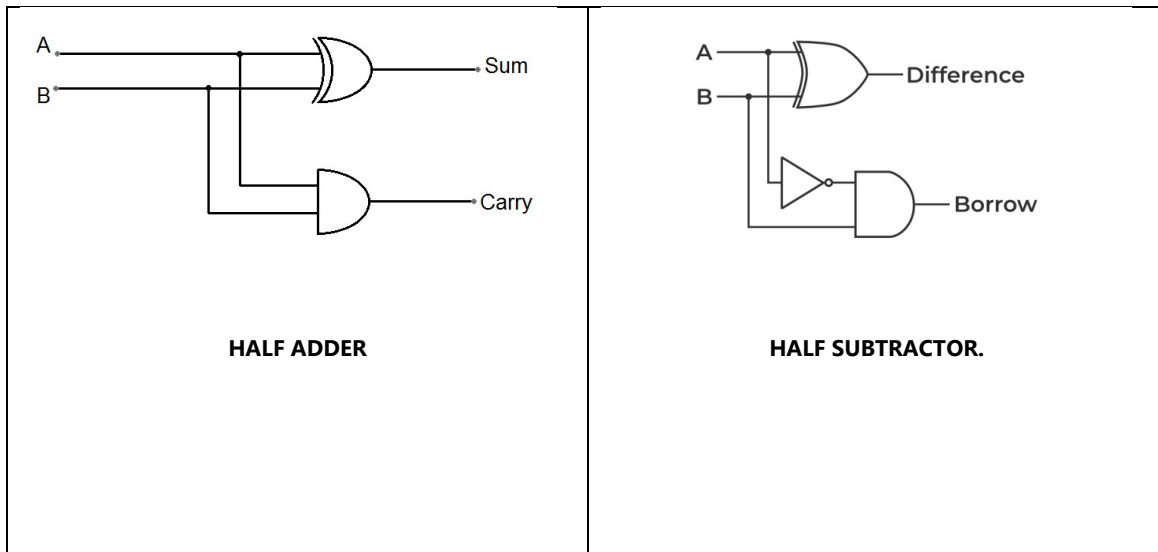
**Pre Lab/ Prior Concept:** Designing of Half adder and subtractor circuits.

---

**Apparatus/TOOL:**

- PC
  - VIVADO TOOL
  - SPARTABN-7 FPGA Board.
-

**CIRCUIT DIAGRAM.**



**VERILOG -CODE.**

**1. Half Adder**

**Gate-Level Modeling**

**Code**

```
module Half_Adder_Gate_Level(output Sum, Carry, input A, B);
  xor (Sum, A, B); // Sum = A XOR B
  and (Carry, A, B); // Carry = A AND B
endmodule
```

**Test Bench.**

```
module Half_Adder_Gate_Level_tb;
  reg A, B;
  wire Sum, Carry;

  // Instantiate the Half Adder module
  Half_Adder_Gate_Level uut (.Sum(Sum), .Carry(Carry), .A(A), .B(B));

  initial
  begin
    $monitor("A = %b, B = %b, Sum = %b, Carry = %b", A, B, Sum, Carry);
    A = 0; B = 0; #10;
  end
endmodule
```

**Department of Electronics and Telecommunication Engineering**  
**(NBA ACCREDITED)**  
**Digital System Design Laboratory**  
**Academic Year 2025-2026, SEM-5, Odd Semester**

```
A = 0; B = 1; #10;  
A = 1; B = 0; #10;  
A = 1; B = 1; #10;  
$stop;  
end  
endmodule
```

---

### **Dataflow Modeling**

#### **Code**

```
module Half_Adder_Dataflow(output Sum, Carry, input A, B);  
    assign Sum = A ^ B; // Sum = A XOR B  
    assign Carry = A & B; // Carry = A AND B  
endmodule
```

#### **Test Bench**

```
verilog  
CopyEdit  
module Half_Adder_Dataflow_tb;  
    reg A, B;  
    wire Sum, Carry;  
  
    // Instantiate the Half Adder module  
    Half_Adder_Dataflow uut (.Sum(Sum), .Carry(Carry), .A(A), .B(B));  
  
    initial  
    begin  
        $monitor("A = %b, B = %b, Sum = %b, Carry = %b", A, B, Sum, Carry);  
        A = 0; B = 0; #10;  
        A = 0; B = 1; #10;  
        A = 1; B = 0; #10;  
        A = 1; B = 1; #10;  
        $stop;  
    end  
endmodule
```

---

### **Behavioral Modeling**

#### **Code**

```
module Half_Adder_Behavioral(output reg Sum, Carry, input A, B);  
    always @(*) begin  
        Sum = A ^ B; // Sum = A XOR B  
        Carry = A & B; // Carry = A AND B  
    end  
endmodule
```

#### **Test Bench**

**Department of Electronics and Telecommunication Engineering**  
**(NBA ACCREDITED)**  
**Digital System Design Laboratory**  
**Academic Year 2025-2026, SEM-5, Odd Semester**

```
module Half_Adder_Behavioral_tb;
    reg A, B;
    wire Sum, Carry;

    // Instantiate the Half Adder module
    Half_Adder_Behavioral uut (.Sum(Sum), .Carry(Carry), .A(A), .B(B));

    initial begin
        $monitor("A = %b, B = %b, Sum = %b, Carry = %b", A, B, Sum, Carry);
        A = 0; B = 0; #10;
        A = 0; B = 1; #10;
        A = 1; B = 0; #10;
        A = 1; B = 1; #10;
        $stop;
    end
endmodule
```

---

## 2. Half Subtractor

### Gate-Level Modeling

#### Code

```
module Half_Subtractor_Gate_Level(output Diff, Borrow, input A, B);
    xor (Diff, A, B); // Difference = A XOR B
    and (Borrow, ~A, B); // Borrow = NOT A AND B
endmodule
```

#### Test Bench

```
module Half_Subtractor_Gate_Level_tb;
    reg A, B;
    wire Diff, Borrow;

    // Instantiate the Half Subtractor module
    Half_Subtractor_Gate_Level uut (.Diff(Diff), .Borrow(Borrow), .A(A), .B(B));

    initial begin
        $monitor("A = %b, B = %b, Diff = %b, Borrow = %b", A, B, Diff, Borrow);
        A = 0; B = 0; #10;
        A = 0; B = 1; #10;
        A = 1; B = 0; #10;
        A = 1; B = 1; #10;
        $stop;
    end
endmodule
```

---

### Dataflow Modeling

#### Code

**Department of Electronics and Telecommunication Engineering**  
**(NBA ACCREDITED)**  
**Digital System Design Laboratory**  
**Academic Year 2025-2026, SEM-5, Odd Semester**

```
module Half_Subtractor_Dataflow(output Diff, Borrow, input A, B);  
    assign Diff = A ^ B;    // Difference = A XOR B  
    assign Borrow = ~A & B; // Borrow = NOT A AND B  
endmodule
```

### Test Bench

```
module Half_Subtractor_Dataflow_tb;  
    reg A, B;  
    wire Diff, Borrow;  
  
    // Instantiate the Half Subtractor module  
    Half_Subtractor_Dataflow uut (.Diff(Diff), .Borrow(Borrow), .A(A), .B(B));  
  
    initial begin  
        $monitor("A = %b, B = %b, Diff = %b, Borrow = %b", A, B, Diff, Borrow);  
        A = 0; B = 0; #10;  
        A = 0; B = 1; #10;  
        A = 1; B = 0; #10;  
        A = 1; B = 1; #10;  
        $stop;  
    end  
endmodule
```

---

### Behavioral Modeling

#### Code

```
module Half_Subtractor_Behavioral(output reg Diff, Borrow, input A, B);  
    always @(*) begin  
        Diff = A ^ B;    // Difference = A XOR B  
        Borrow = ~A & B; // Borrow = NOT A AND B  
    end  
endmodule
```

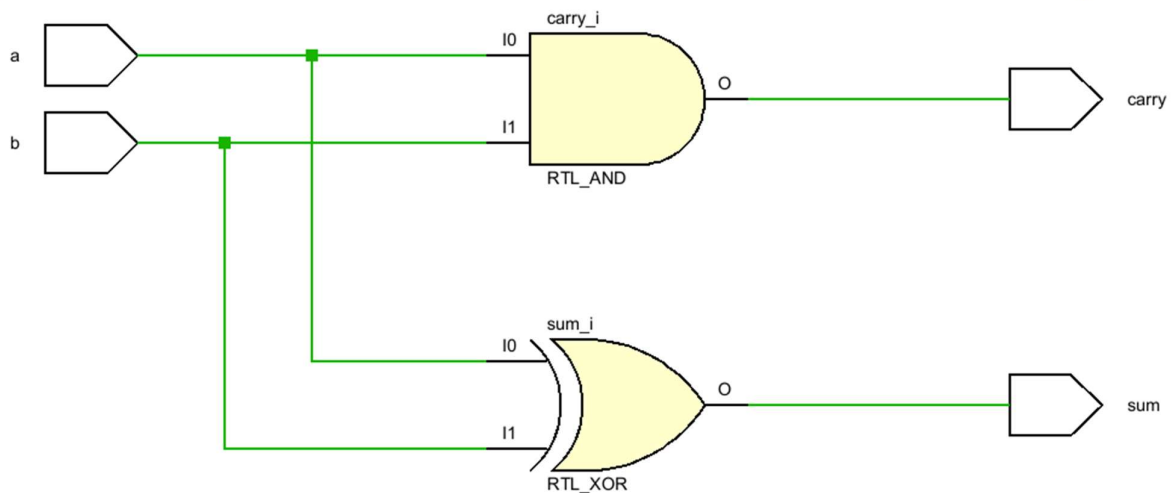
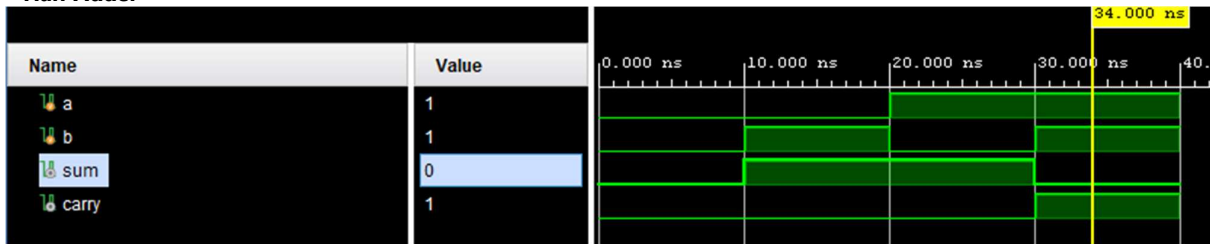
#### Test Bench

```
module Half_Subtractor_Behavioral_tb;  
    reg A, B;  
    wire Diff, Borrow;  
  
    // Instantiate the Half Subtractor module  
    Half_Subtractor_Behavioral uut (.Diff(Diff), .Borrow(Borrow), .A(A), .B(B));  
  
    initial begin  
        $monitor("A = %b, B = %b, Diff = %b, Borrow = %b", A, B, Diff, Borrow);  
        A = 0; B = 0; #10;  
        A = 0; B = 1; #10;  
        A = 1; B = 0; #10;  
    end  
endmodule
```

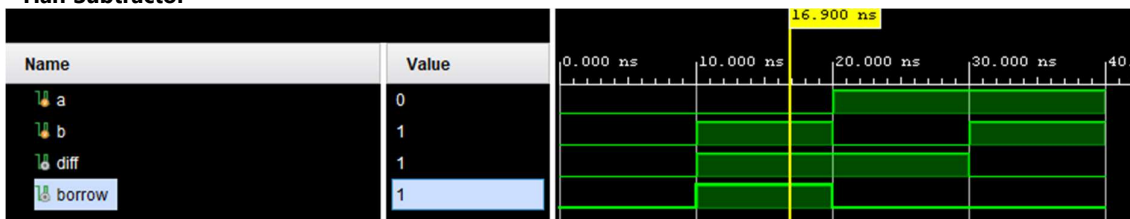
```
A = 1; B = 1; #10;
$stop;
end
endmodule
```

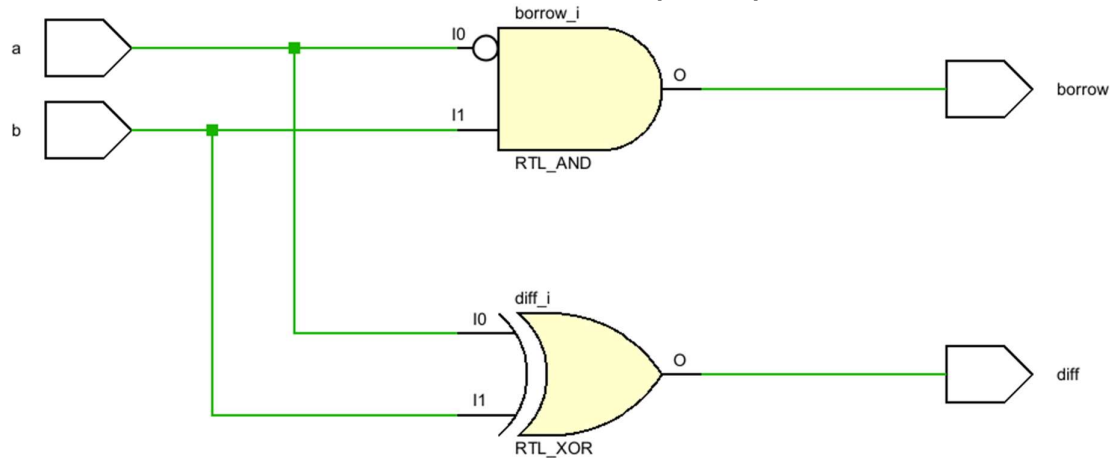
### SIMULATION RESULTS.

#### Half Adder



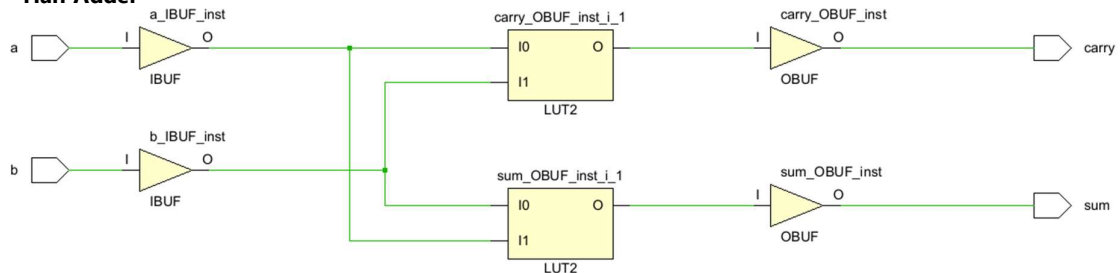
#### Half Subtractor



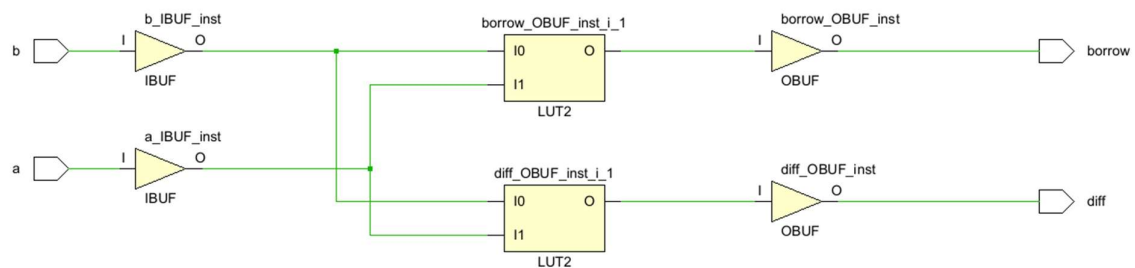


#### SYNTHESIS MODEL.

##### Half Adder



##### Half Subtractor



#### CONCLUSION.

The Half Adder and Half Subtractor were successfully realized on the FPGA. The Half Adder generated the Sum and Carry outputs using XOR and AND gates, while the Half Subtractor produced the Difference and Borrow using the same gates. Both circuits were verified through testbenches, and the observed outputs matched the expected truth table results, confirming correct functionality. This implementation showcased the efficient utilization of basic logic gates on the FPGA.

#### POST LAB QUESTION.

**Department of Electronics and Telecommunication Engineering  
(NBA ACCREDITED)  
Digital System Design Laboratory  
Academic Year 2025-2026, SEM-5, Odd Semester**

**1. Derive the Boolean equation of sum and carry for half adder.**

Half Adder:

A half adder adds two single-bit binary numbers and produces two outputs:

- Sum (S)
- Carry (C)

| A | B | Sum (S) | Carry (C) |
|---|---|---------|-----------|
| 0 | 0 | 0       | 0         |
| 0 | 1 | 1       | 0         |
| 1 | 0 | 1       | 0         |
| 1 | 1 | 0       | 1         |

Sum (S):

The sum output of a half adder is 1 if either A or B is 1, but not both. This is essentially the XOR (exclusive OR) of A and B.

$$S = A \oplus B$$

Carry (C):

The carry output of a half adder is 1 only when both A and B are 1, which is the AND operation of A and B.

$$C = A \cdot B$$

So, the Boolean equations for the half adder are:

$$S = A \oplus B$$

$$C = A \cdot B$$

**2. Derive the Boolean equation of borrow and difference for Half subtractor.**

Half Subtractor:

A half subtractor subtracts two single-bit binary numbers and produces two outputs

- Difference (D)
- Borrow (B)

| A | B | Difference (D) | Borrow (B) |
|---|---|----------------|------------|
| 0 | 0 | 0              | 0          |
| 0 | 1 | 1              | 1          |
| 1 | 0 | 1              | 0          |
| 1 | 1 | 0              | 0          |

**1. Difference (D):**

The difference output of a half subtractor is 1 when A is 1 and B is 0, or when A and B are different. This is essentially the **XOR** of A and B.

$$D = A \oplus B$$

**2. Borrow (B):**

The borrow output of a half subtractor is 1 when A is 0 and B is 1 (i.e., when A cannot subtract B). This is the NOT A AND B operation.

$$B = A' \cdot B$$

So, the Boolean equations for the half subtractor are:

- $D = A \oplus B$

- $B = A' \cdot B$



**Department of Electronics and Telecommunication Engineering**  
**(NBA ACCREDITED)**  
**Digital System Design Laboratory**  
**Academic Year 2025-2026, SEM-5, Odd Semester**

**ASSESSMENT RECORD.**

| <b>Practical<br/>Performance</b><br>[Model/Code,<br>Results]<br><b>(5 Marks)</b> | <b>Presentation/<br/>Learning<br/>Outcome</b><br>[Observation,<br>Conclusion] <b>(5<br/>Marks)</b> | <b>Total (10 Marks)</b> | <b>Sign</b> |
|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|-------------------------|-------------|
|                                                                                  |                                                                                                    |                         |             |

